

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ
МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №1

“Методы сортировки”

Вариант 1 / 3 / 2 + 5

Выполнила:
студентка 119 группы
Учар А. Х.

Преподаватель:
Журихин Д. М.

Москва

2023

Содержание

1. Постановка задачи.....	2
2. Результаты экспериментов	3
3. Структура программы и спецификация функций	6
4. Отладка программы, тестирование функций	9
5. Анализ допущенных ошибок	10
Список используемой литературы	11

1. Постановка задачи

Необходимо реализовать два метода сортировки массива чисел и провести их экспериментальное сравнение путем вычисления количества сравнений и перемещений элементов массива для каждого метода. Производится сравнение таких алгоритмов сортировки, как метод простого выбора и пирамидальная сортировка. Элементы массивов имеют тип `int`, а числа необходимо упорядочить по неубыванию их модулей (при сравнении элементов не учитывается знак).

2. Результаты экспериментов

В приведенных ниже таблицах номер сгенерированного массива означает:

- 1 – Элементы массива уже упорядочены в нужном порядке
- 2 – Элементы массива упорядочены в обратном порядке
- 3, 4 – Расстановка элементов случайна

n – число элементов в массиве.

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	45	45	45	45	45
100	4950	4950	4950	4950	4950
1000	499500	499500	499500	499500	499500
10000	49995000	49995000	49995000	49995000	49995000

Таблица 1: Количество сравнений при сортировке методом простого выбора

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	0	5	6	6	4,3
100	0	50	94	94	59,5
1000	0	500	990	990	620,0
10000	0	5000	9995	9987	6245,5

Таблица 2: Количество перемещений при сортировке методом простого выбора

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	41	35	37	37	37,5
100	1081	944	1043	1043	1027,8
1000	17583	15965	16862	16862	16818,0
10000	244460	226682	235188	235473	235450,8

Таблица 3: Количество сравнений при пирамидальной сортировке

n	Номер сгенерированного массива				Среднее значение
	1	2	3	4	
10	31	22	29	29	27,8
100	641	517	595	595	587,0
1000	9709	8317	9109	9109	9061,0
10000	131957	116697	124134	124188	124244,0

Таблица 4: Количество перемещений при пирамидальной сортировке

Оценки сложности

Сортировка методом простого выбора

Пусть число i показывает, в какой раз мы проходимся по массиву. Тогда на i -том шаге будет выполнено $n-i$ сравнений и максимум 1 перемещение элементов. В сумме количество сравнений можно найти по формуле

$$\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$$

А количество перемещений в худшем случае равно $n-1$. Следовательно сложность алгоритма - $O(n^2)$ (Подробнее в [1]).

Пирамидальная сортировка

Количество сравнений в пирамидальной сортировке равно $O(n \log n)$, так как в худшем случае каждый элемент сравнивается с $\log n$ другими элементами.

Количество перемещений также равно $O(n \log n)$, так как каждый элемент перемещается не более чем $\log n$ раз. Таким образом, пирамидальная сортировка имеет общую сложность $O(n \log n)$ (Подробнее в [2]).

Учитывая небольшое количество тестов и тот факт, что результаты соответствуют выведенным формулам, можно заключить, что полученные результаты эксперимента соответствуют теоретической оценке.

При сортировке методом простого выбора выполняется значительно меньше обменов значений элементов, чем при пирамидальной сортировке.

Пирамидальная сортировка имеет недостаточно высокую производительность при небольших значениях n . Сортировка методом простого выбора при таких значениях работает лучше.

Число сравнений в сортировке методом простого выбора не зависит от начального порядка элементов, а зависит лишь от размера массива. Поэтому

сложность данного алгоритма при любых входных данных будет $O(n^2)$. В то время как пирамидальная сортировка имеет доказанную оценку худшего случая $O(n \log n)$. Следовательно пирамидальная сортировка эффективнее при больших n , что можно увидеть и на приведенных тестах.

Пирамидальная сортировка работает на почти отсортированных массивах столь же долго, как и на хаотических данных. А при методе сортировки простым выбором на почти отсортированном массиве производится наименьшее количество перемещений.

В заключение, сортировка методом простого выбора эффективнее при малых n , при средних и больших же лучше использовать пирамидальную сортировку.

3. Структура программы и спецификация функций

Функции в программе: `power_of_minus_one`, `NonDecArray`, `NonIncArray`, `RandArray`, `compare`, `swap`, `SelectionSort`, `heapify`, `HeapSort`, `abscmp`, `cmp`, `test`, `print`

1. `static int power_of_minus_one(int n)`

Параметры:

`int n` – степень, в которую нужно возвести -1

Функционирование:

Возводит -1 в нужную степень

2. `static void NonDecArray(int n, int *a)`

Параметры:

`int n` – размер массива, `int *a` – указатель на массив из целых чисел

Функционирование:

Генерирует неубывающий массив. Для этого использует функцию `srand` из библиотеки `stdlib.h`. Чтобы получить неубывающие элементы, каждый раз меняет интервал для генерации числа.

3. `static void NonIncArray(int n, int *a)`

Параметры:

`int n` – размер массива, `int *a` – указатель на массив из целых чисел

Функционирование:

Генерирует невозрастающий массив. Для этого использует функцию `void srand(unsigned int seed)` из библиотеки `stdlib.h`. Чтобы получить невозрастающие элементы, каждый раз меняет интервал для генерации числа.

4. `static void RandArray(int n, int *a)`

Параметры:

`int n` – размер массива, `int *a` – указатель на массив из целых чисел

Функционирование:

Генерирует произвольный массив. Для этого использует функцию `void srand(unsigned int seed)` из библиотеки `stdlib.h`.

5. `static int compare(int x, int y)`

Параметры:

`int x`, `int y` – числа для сравнения

Функционирование:

Возвращает 1, если модуль числа `x` больше модуля числа `y`,

увеличивает счетчик сравнений (прибавляет 1 к глобальной переменной `comparisons`).

6. `static void swap(int *a, int i, int j)`

Параметры:

`int *a` – указатель на массив целых чисел; `int i, int j` – индексы элементов, которые нужно поменять местами

Функционирование:

Меняет местами элементы с индексами `i` и `j`. Увеличивает счетчик перемещений (прибавляет 1 к глобальной переменной `moves`).

7. `static void SelectionSort(int n, int *a)`

Параметры:

`int n` – количество элементов массива, `int *a` – указатель на массив целых чисел

Функционирование:

Производит сортировку методом простого выбора. Находится номер минимального значения в неотсортированной части и производится обмен этого значения со значением первой неотсортированной позиции (обмен не нужен, если минимальный элемент уже находится на данной позиции). Происходит вызов функций `compare` и `swap`.

8. `static void heapify(int n, int *a, int i)`

Параметры:

`int n` – количество элементов массива, `int *a` – указатель на массив целых чисел, `int i` – индекс корня текущего поддерева

Функционирование:

Вспомогательная функция для реализации пирамидальной сортировки. Сравнивает значения родительского элемента и двух детей, перемещает наибольшее значение в корень. Рекурсивно вызывается эта же функция, чтобы теперь отсортировать часть дерева, которая была изменена. Использует функцию `swap`.

9. `static void HeapSort(int n, int *a)`

Параметры:

`int n` – количество элементов массива, `int *a` – указатель на массив целых чисел

Функционирование:

Производит пирамидальную сортировку, вызывая вспомогательную функцию `heapify`. Распределяет элементы в нужном порядке и «удаляет» элементы, перемещая текущий корень в самый конец массива. Использует функцию `swap`.

10. `static int abscmp(const void *a, const void *b)`

Параметры:

`const void *a, const void *b` – элементы неопределённого вида для сравнения

Функционирование:

Возвращает разность модулей чисел `a` и `b`, приведя их к типу `int` (функция нужна для использования `qsort` в тестирующей функции)

11. `static int cmp(const void *a, const void *b)`

Параметры:

`const void *a, const void *b` – элементы неопределённого вида для сравнения

Функционирование:

Возвращает разность чисел `a` и `b`, приведя их к типу `int` (функция нужна для использования `qsort` в тестирующей функции)

12. `static int test(int *arr, int *tested_arr, int n)`

Параметры:

`int *arr` – указатель на массив `arr`, `int *tested_arr` – указатель на массив `tested_arr`, `int n` – количество элементов в массивах

Функционирование:

Проверяет, правильно ли отсортирован массив `tested_arr`.

Возвращает 1, если массив был отсортирован (по модулям)

неправильно. Возвращает 2, если массив `tested_arr` был как-то изменен. Сортирует алгебраические значения элементов. Сравнивая массивы `arr` и `tested_arr`, понимаем, мог ли массив `tested_arr` измениться, могли ли в него добавиться новые элементы.

Возвращает 0, если массив отсортирован правильно.

13. `static void print(int status, int n, int *a, int *copy, int *testcopy1, int *testcopy2)`

Параметры:

`int status` – отвечает за вид массива, который нужно сгенерировать, `int n` – количество элементов массива, `int *a` – указатель на массив целых чисел для сортировки выбором,

`int *copy` – указатель копии массива `a` для пирамидальной сортировки, `int *testcopy1` – указатель копии массива `a` для тестирующей функции для сортировки выбором, `int *testcopy2` – указатель копии массива `a` для тестирующей функции для пирамидальной сортировки.

Функционирование:

Производит вывод результатов для соответствующего вида сгенерированного массива.

4. Отладка программы, тестирование функций

1. Генерирование массивов различных длин при помощи вызова функций `NonDecArray`, `NonIncArray`, `RandArray` и их вывод на экран
2. Тестирование функции для сортировки методом простого выбора `SelectionSort` на заданных массивах (проверка вручную) и при помощи тестирующей функции `test`
3. Тестирование функций для пирамидальной сортировки `heapify` и `HeapSort` на заданных массивах (проверка вручную) и при помощи тестирующей функции `test`
4. Тестирование функции `test` на произвольных массивах одинаковой длины

5. Анализ допущенных ошибок

1. Сначала функции `SelectionSort` и `HeapSort` сортировали алгебраические значения элементов, отдельно производилось изменение всех элементов массива на их модули. Затем эти функции были изменены, они сортируют абсолютные значения элементов, не меняя сам массив
2. Тестирующая функция в начале не рассматривала отдельные виды ошибок и не учитывала случай, когда при правильной сортировке модулей элементов, алгебраические значения чисел изменились. Функция была исправлена добавлением двух видов ошибок. В первом случае сортируются алгебраические значения элементов (тем самым мы точно понимаем, что никакие элементы массива `tested_arr` не отличаются от элементов массива `arr`). Во втором случае сортируются абсолютные значения, что и требуется в задании.
3. Переменные `moves` и `comparisons` изначально были локальными, из-за чего приходилось передавать их в каждую функцию, поэтому они стали глобальными.

Список используемой литературы

- [1] Лорин Г. Сортировка и системы сортировки. – М.: Наука, 1983, Москва.
- [2] Кормен Т., Лейзерсон Ч., Ривест Р, Штайн К. Алгоритмы: построение и анализ. Второе издание. — М.:«Вильямс», 2005.